

Chapter 7

Adversarial Examples in Specific Domains

PCC190 – Secure Machine Learning
Prof. Rodrigo C. P. Silva – DECOM/UFOP

October 2, 2026

Contents

From ℓ_p balls to real constraints	1
1 Adversarial Attacks on Computer Vision Models	1
1.1 Beyond classification	1
1.2 Patch attacks	2
2 Generating Adversarial Text for NLP Models	2
2.1 What makes text different	2
2.2 Anatomy of a text attack	2
2.3 Large language models	3
3 Attacks on Reinforcement Learning Agents	3
4 Physical Adversarial Attacks	3
5 Domain-Specific Attack Considerations	4
6 Practice: Generating Adversarial Text	4
6.1 Using TextAttack	4
6.2 A minimal greedy synonym attack from scratch	5
6.3 Suggested experiments	5
Exercises	5

From ℓ_p balls to real constraints

The previous chapters used a continuous input space and an ℓ_p threat set. Outside image classification benchmarks both assumptions break. Text is discrete; an agent’s inputs depend on its own past actions; a physical object is seen through a camera under changing conditions; a malware binary must still execute. The general problem is

$$\max_{\mathbf{x}' \in \mathcal{T}(\mathbf{x})} \mathcal{L}(f(\mathbf{x}'), y) \quad \text{or} \quad \text{find } \mathbf{x}' \in \mathcal{T}(\mathbf{x}) \text{ with } f(\mathbf{x}') \neq y, \quad (1)$$

where $\mathcal{T}(\mathbf{x})$ is the set of *admissible transformations* of $\mathbf{x}$: those that preserve its meaning, label, or function. The domain dictates $\mathcal{T}$, and $\mathcal{T}$ dictates which optimization methods are applicable. [Pierazzi et al. \(2020\)](#) call this the distinction between the *feature space*, where the model operates, and the *problem space*, where real objects live.

1 Adversarial Attacks on Computer Vision Models

1.1 Beyond classification

Object detection and segmentation. A detector outputs many predictions per image (boxes, classes, scores). Attacks define losses over the set of proposals: to make an object disappear, push all proposals overlapping it toward background; to create a false object, raise a target class score in a chosen region. Dense Adversary Generation ([Xie et al., 2017](#)) optimizes the sum of per-proposal (or per-pixel, for segmentation) target losses over the set of still-correct predictions, removing them from the set as they flip.

Universal perturbations. Moosavi-Dezfooli et al. (2017) find a single δ with $\|\delta\|_p \leq \epsilon$ that fools the model on most inputs from $\mathcal{D}$, by iterating over training images and accumulating minimal perturbations (DeepFool steps) projected back onto the ball. Their existence suggests that decision boundaries near natural images share common normal directions.

Non- ℓ_p perturbations. Rotations and translations (Engstrom et al., 2019), color and texture changes, and perceptual-metric-bounded perturbations (Laidlaw et al., 2021) model changes that humans ignore but ℓ_p does not capture. A grid search over a few rotation angles and shifts already fools many standard and ℓ_p -robust models.

1.2 Patch attacks

A patch attack replaces a region of the image instead of adding small noise everywhere. With a mask $\mathbf{M}$ placed by a location transform A :

$$\mathbf{x}' = (\mathbf{1} - A(\mathbf{M})) \odot \mathbf{x} + A(\mathbf{M}) \odot A(\mathbf{p}).$$

The patch $\mathbf{p}$ is unconstrained in value but limited in area, so it is visible. Brown et al. (2017) trained *universal*, location-independent patches that make classifiers output a target class, using expectation over locations, scales and rotations. Patches bridge the digital and physical settings (Section 4).

2 Generating Adversarial Text for NLP Models

2.1 What makes text different

- **Discreteness.** Inputs are token sequences; gradients with respect to embeddings do not correspond to valid tokens.
- **Perceptibility.** Any change is visible. The constraint is not invisibility but *preservation of meaning and fluency*, and a small edit (“not”) can flip the true label.
- **Evaluation.** Whether an example is valid requires human or model judgment of semantics and grammar.

2.2 Anatomy of a text attack

Morris et al. (2020b) decompose every text attack into four components:

1. **Goal function:** untargeted classification, targeted classification, or a sequence-level goal (change a translation, inject a phrase).
2. **Transformation:** how candidates are generated from the current text. Character level (typos, homoglyphs, swaps; Li et al., 2019), word level (synonyms from counter-fitted embeddings or WordNet; masked-language-model suggestions, Li et al., 2020), sentence level (paraphrase, distractor sentences, Jia and Liang, 2017).
3. **Constraints:** limits that keep the example valid: maximum fraction of words changed, part-of-speech consistency, minimum sentence-encoder similarity, grammaticality checks, no edits to stopwords.
4. **Search method:** greedy with word importance ranking, beam search, genetic algorithms (Alzantot et al., 2018), or gradient-guided search.

TextFooler as an example. Jin et al. (2020):

1. rank words by importance: the drop in the true-class score when the word is deleted;
2. for each word in order, gather the nearest synonyms in a counter-fitted embedding space, keep those with the same part of speech and with sentence-encoder similarity above a threshold;
3. replace the word with the candidate that most reduces the true-class score; stop as soon as the label flips.

Gradient-guided token search. HotFlip (Ebrahimi et al., 2018) uses the first-order approximation of the loss change when swapping token a for token b at position i :

$$\Delta\mathcal{L}_{i,a \rightarrow b} \approx (\mathbf{e}_b - \mathbf{e}_a)^\top \nabla_{\mathbf{e}_i} \mathcal{L},$$

where $\mathbf{e}$ are embeddings. Ranking all (i, b) by this quantity costs one backward pass. Wallace et al. (2019) use the same approximation to find *universal triggers*: short token sequences that, prepended to any input, push the model toward a target behavior.

2.3 Large language models

Aligned LLMs are attacked with the same tools at larger scale. Greedy Coordinate Gradient (Zou et al., 2023) combines HotFlip-style gradient ranking with exhaustive evaluation of a sampled candidate set to optimize an adversarial suffix that makes the model begin its answer with an attacker-chosen string; suffixes found on open models partly transfer to closed ones. A distinct threat is *indirect prompt injection* (Greshake et al., 2023): instructions embedded in retrieved documents, web pages or tool outputs that the model treats as commands. Here the “perturbation” is any text the attacker can place in the model’s context, and the goal is to hijack actions rather than to flip a label.

Validity of adversarial text

Morris et al. (2020a) found that many examples produced by popular word-substitution attacks did not preserve meaning or were ungrammatical when judged by humans; tightening constraints to human-acceptable levels reduced attack success substantially. Always inspect examples and report the constraints used.

3 Attacks on Reinforcement Learning Agents

An agent with policy $\pi(a | s)$ interacts with an MDP. The attacker can act on several channels:

Observation perturbations. The attacker perturbs the agent’s observation s_t to $\tilde{s}_t \in \mathcal{B}_p(s_t, \epsilon)$, while the true state is unchanged. Huang et al. (2017) applied FGSM to Atari policies, treating the policy’s preferred action as the label, and showed large drops in reward. Two refinements matter:

- *Timing*: attack only at critical states, for example when the gap between the most and least preferred action probabilities is large (strategically-timed attack, Lin et al., 2017), which reaches similar damage with far fewer perturbed steps.
- *Planning*: lure the agent to a target state by predicting future states and choosing perturbations that steer it there (Lin et al., 2017).

Zhang et al. (2020) formalize this as a state-adversarial MDP and show that the optimal adversary can be found by solving another MDP, and that robust training must regularize the policy’s sensitivity to observation perturbations.

Adversarial policies. In multi-agent environments, the attacker controls another agent and cannot touch the victim’s inputs directly. Gleave et al. (2020) trained opponent policies with standard RL that defeat strong victims in simulated competitive games by producing unusual, seemingly random behavior that pushes the victim’s observations off-distribution. Nothing about the attack is small or imperceptible; it is natural within the environment.

Other channels. Perturbing rewards during training (a form of poisoning), perturbing dynamics (robust adversarial RL trains against an adversary applying forces, Pinto et al., 2017), and backdooring policies with triggers in the observation.

4 Physical Adversarial Attacks

A physical attack must survive the pipeline: fabrication (printing, 3D printing), viewing conditions (distance, angle, lighting), camera capture (noise, blur, compression) and preprocessing. Three ideas make this possible.

Expectation over Transformation. Optimize the expected loss over a distribution $\mathcal{T}$ of physical transformations (Athalye et al., 2018):

$$\max_{\mathbf{x}'} \mathbb{E}_{t \sim \mathcal{T}} [\mathcal{L}(f(t(\mathbf{x}')), y)] - \lambda \mathbb{E}_{t \sim \mathcal{T}} [d(t(\mathbf{x}'), t(\mathbf{x}))].$$

With $\mathcal{T}$ covering rotation, scale, translation, lighting and noise (and, for 3D objects, rendering from different viewpoints), the result is robust enough to be printed or 3D-printed and still fool a classifier from many angles.

Printability and smoothness. A non-printability score penalizes colors the printer cannot reproduce, and total variation encourages smooth patterns that survive camera blur (Sharif et al., 2016).

Localized artifacts. Restricting the perturbation to an object the attacker can plausibly wear or attach: eyeglass frames that cause face recognition to misidentify or impersonate (Sharif et al., 2016); black-and-white stickers on stop signs (RP₂), optimized with masks and real photographs under varying conditions (Eykholt et al., 2018); printed patches that hide people from detectors (Thys et al., 2019). Over-the-air audio attacks face analogous constraints from room acoustics, although Carlini and Wagner (2018) demonstrated their targeted speech-to-text attack digitally.

Evaluation. Physical results should be reported over many captured frames, distances and angles, as the fraction of frames fooled, and compared with a control artifact (random pattern of the same size and location).

5 Domain-Specific Attack Considerations

Malware detection. Features (API calls, permissions, byte n-grams) are extracted from programs. Modifications must keep the program functional and plausible. Typical constraints allow adding features (dead code, unused permissions, padding bytes, new sections) but not removing those required for malicious behavior (Grosse et al., 2017). A feature-space adversarial example may not correspond to any program; problem-space attacks (Pierazzi et al., 2020) work with transplanted code gadgets, which must preserve semantics, be robust to preprocessing, and remain plausible to analysts. The attacker’s capability set is *asymmetric* and discrete.

Tabular data. Features are mixed (categorical, ordinal, continuous), have domain ranges and dependencies (age vs. years of employment), and some are immutable or costly to change (in credit scoring, an applicant can change the requested amount but not their history). ℓ_p distance on normalized features is a poor proxy for perceptibility or cost; attacks weight features by importance and plausibility (Ballet et al., 2019) and project onto feasibility constraints. Explainability methods often serve as attack guides here.

Medical imaging. Perturbations can be invisible and incentives exist (insurance reimbursement, clinical trial outcomes); Finlayson et al. (2019) discuss these scenarios. Standardized acquisition pipelines may make digital attacks easier to deliver than in natural images.

Graphs. In node classification, the attacker can add or remove edges or modify node features. Perturbations are discrete and their effect propagates through message passing; Nettack (Zügner et al., 2018) perturbs the graph structure and features while preserving degree distribution statistics so the attack is less noticeable.

Questions to ask in any new domain

1. What is the problem space, and what transformations preserve label, meaning or function?
2. Can the attacker act on the raw object, the features, or only the environment?
3. Is the search space continuous, discrete, or mixed, and which optimizer fits it?
4. What does a defender or human reviewer see, and what would look suspicious?
5. What is the cost of each modification, and what is the attacker’s budget?

6 Practice: Generating Adversarial Text

6.1 Using TextAttack

```
# pip install textattack transformers
import transformers, textattack
from textattack.models.wrappers import HuggingFaceModelWrapper
from textattack.attack_recipes import TextFoolerJin2019
from textattack.datasets import HuggingFaceDataset
from textattack import Attacker, AttackArgs

name = "textattack/bert-base-uncased-imdb"
model = transformers.AutoModelForSequenceClassification.from_pretrained(name)
tok = transformers.AutoTokenizer.from_pretrained(name)
wrapper = HuggingFaceModelWrapper(model, tok)

attack = TextFoolerJin2019.build(wrapper)
dataset = HuggingFaceDataset("imdb", split="test")
args = AttackArgs(num_examples=100, log_to_csv="textfooler_imdb.csv",
                  disable_stdout=True, random_seed=0)
Attacker(attack, dataset, args).attack_dataset()
```

TextAttack prints success rate, average perturbed word percentage and average number of queries. Other recipes (BAEGarg2019, BERTAttackLi2020, DeepWordBugGao2018, PWSRen2019) can be swapped in for comparison.

6.2 A minimal greedy synonym attack from scratch

```
import torch

@torch.no_grad()
def prob_true(model, tok, texts, y):
    enc = tok(texts, return_tensors="pt", padding=True, truncation=True)
    return torch.softmax(model(**enc).logits, -1)[:, y]

def greedy_synonym_attack(model, tok, text, y, synonyms, max_frac=0.2):
    # synonyms: dict word -> list of candidate replacements
    words = text.split()
    base = prob_true(model, tok, [text], y).item()
    # 1) importance = drop in p(y) when the word is removed
    deleted = [" ".join(words[:i] + words[i+1:]) for i in range(len(words))]
    imp = base - prob_true(model, tok, deleted, y)
    order = imp.argsort(descending=True).tolist()
    changed, budget = 0, int(max_frac * len(words))
    for i in order:
        cands = synonyms.get(words[i].lower(), [])
        if not cands or changed >= budget:
            continue
        trials = [" ".join(words[:i] + [c] + words[i+1:]) for c in cands]
        p = prob_true(model, tok, trials, y)
        j = p.argmax().item()
        if p[j] < prob_true(model, tok, [" ".join(words)], y).item():
            words[i] = cands[j]; changed += 1
        enc = tok([" ".join(words)], return_tensors="pt")
        if model(**enc).logits.argmax(-1).item() != y:
            return " ".join(words), changed
    return None, changed # attack failed within budget
```

The synonym dictionary can be built from WordNet or from nearest neighbors in counter-fitted word vectors. As written, the attack has no semantic or grammatical constraints; adding a sentence-encoder similarity check is Experiment 2.

6.3 Suggested experiments

1. Run TextFooler, BAE and DeepWordBug on 200 IMDB or SST-2 examples against a fine-tuned BERT. Compare success rate, perturbed-word percentage and queries.
2. Add a constraint to the greedy attack: cosine similarity of Universal Sentence Encoder (or any sentence-embedding model) embeddings above a threshold, trying 0.7, 0.8 and 0.9. How does success change?
3. Manually label 30 successful examples from each attack as *valid* (meaning and label preserved, fluent) or not. Report the corrected success rate.
4. Train with adversarial data augmentation (TextAttack's `augment` or adversarial training) and re-evaluate. Does robustness transfer across attack recipes?
5. (Vision) Train a universal adversarial patch on ImageNet with EOT over scale and location, print it, and measure the fraction of frames fooled on a phone camera feed.

Exercises

1. Derive the HotFlip first-order approximation and explain why it can be inaccurate for multi-token edits.
2. For a malware detector over binary features $\mathbf{x} \in \{0, 1\}^d$ where the attacker can only set bits from 0 to 1, write the evasion problem for a linear model $f(\mathbf{x}) = \text{sign}(\mathbf{w}^\top \mathbf{x} + b)$ and give its optimal solution with a budget of k bit flips.
3. Explain why an observation-perturbation attack on an RL agent can achieve large reward loss by perturbing only a few time steps, and how you would choose those steps.
4. Write the EOT objective for a sticker attack on a traffic sign classifier, specifying the mask, the transformation distribution and the printability term.
5. Discuss whether ℓ_p robustness is a meaningful goal for tabular credit-scoring models. Propose a better-suited threat model.

References

- Alzantot, M., Sharma, Y., Elgohary, A., Ho, B.-J., Srivastava, M., and Chang, K.-W. (2018). Generating natural language adversarial examples. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2890–2896.
- Athalye, A., Engstrom, L., Ilyas, A., and Kwok, K. (2018). Synthesizing robust adversarial examples. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 284–293.
- Ballet, V., Renard, X., Aigrain, J., Laugel, T., Frossard, P., and Detyniecki, M. (2019). Imperceptible adversarial attacks on tabular data. *arXiv preprint arXiv:1911.03274*.
- Brown, T. B., Mané, D., Roy, A., Abadi, M., and Gilmer, J. (2017). Adversarial patch. *arXiv preprint arXiv:1712.09665*.
- Carlini, N. and Wagner, D. (2018). Audio adversarial examples: Targeted attacks on speech-to-text. In *IEEE Security and Privacy Workshops (SPW)*, pages 1–7.
- Ebrahimi, J., Rao, A., Lowd, D., and Dou, D. (2018). HotFlip: White-box adversarial examples for text classification. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 31–36.
- Engstrom, L., Tran, B., Tsipras, D., Schmidt, L., and Madry, A. (2019). Exploring the landscape of spatial robustness. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 1802–1811.
- Eykholt, K., Evtimov, I., Fernandes, E., Li, B., Rahmati, A., Xiao, C., Prakash, A., Kohno, T., and Song, D. (2018). Robust physical-world attacks on deep learning visual classification. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1625–1634.
- Finlayson, S. G., Bowers, J. D., Ito, J., Zittrain, J. L., Beam, A. L., and Kohane, I. S. (2019). Adversarial attacks on medical machine learning. *Science*, 363(6433):1287–1289.
- Gleave, A., Dennis, M., Wild, C., Kant, N., Levine, S., and Russell, S. (2020). Adversarial policies: Attacking deep reinforcement learning. In *International Conference on Learning Representations (ICLR)*.
- Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., and Fritz, M. (2023). Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 79–90.
- Grosse, K., Papernot, N., Manoharan, P., Backes, M., and McDaniel, P. (2017). Adversarial examples for malware detection. In *European Symposium on Research in Computer Security (ESORICS)*, pages 62–79.
- Huang, S., Papernot, N., Goodfellow, I., Duan, Y., and Abbeel, P. (2017). Adversarial attacks on neural network policies. *arXiv preprint arXiv:1702.02284*.
- Jia, R. and Liang, P. (2017). Adversarial examples for evaluating reading comprehension systems. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2021–2031.
- Jin, D., Jin, Z., Zhou, J. T., and Szolovits, P. (2020). Is BERT really robust? A strong baseline for natural language attack on text classification and entailment. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 8018–8025.
- Laidlaw, C., Singla, S., and Feizi, S. (2021). Perceptual adversarial robustness: Defense against unseen threat models. In *International Conference on Learning Representations (ICLR)*.
- Li, J., Ji, S., Du, T., Li, B., and Wang, T. (2019). TextBugger: Generating adversarial text against real-world applications. In *Network and Distributed System Security Symposium (NDSS)*.
- Li, L., Ma, R., Guo, Q., Xue, X., and Qiu, X. (2020). BERT-ATTACK: Adversarial attack against BERT using BERT. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6193–6202.
- Lin, Y.-C., Hong, Z.-W., Liao, Y.-H., Shih, M.-L., Liu, M.-Y., and Sun, M. (2017). Tactics of adversarial attack on deep reinforcement learning agents. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 3756–3762.
- Moosavi-Dezfooli, S.-M., Fawzi, A., Fawzi, O., and Frossard, P. (2017). Universal adversarial perturbations. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1765–1773.
- Morris, J., Lifland, E., Lanchantin, J., Ji, Y., and Qi, Y. (2020a). Reevaluating adversarial examples in natural language. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 3829–3839.

- Morris, J., Lifland, E., Yoo, J. Y., Grigsby, J., Jin, D., and Qi, Y. (2020b). TextAttack: A framework for adversarial attacks, data augmentation, and adversarial training in NLP. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 119–126.
- Pierazzi, F., Pendlebury, F., Cortellazzi, J., and Cavallaro, L. (2020). Intriguing properties of adversarial ML attacks in the problem space. In *IEEE Symposium on Security and Privacy (S&P)*, pages 1332–1349.
- Pinto, L., Davidson, J., Sukthankar, R., and Gupta, A. (2017). Robust adversarial reinforcement learning. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 2817–2826.
- Sharif, M., Bhagavatula, S., Bauer, L., and Reiter, M. K. (2016). Accessorize to a crime: Real and stealthy attacks on state-of-the-art face recognition. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 1528–1540.
- Thys, S., Van Ranst, W., and Goedemé, T. (2019). Fooling automated surveillance cameras: Adversarial patches to attack person detection. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*.
- Wallace, E., Feng, S., Kandpal, N., Gardner, M., and Singh, S. (2019). Universal adversarial triggers for attacking and analyzing NLP. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, pages 2153–2162.
- Xie, C., Wang, J., Zhang, Z., Zhou, Y., Xie, L., and Yuille, A. (2017). Adversarial examples for semantic segmentation and object detection. In *IEEE International Conference on Computer Vision (ICCV)*, pages 1369–1378.
- Zhang, H., Chen, H., Xiao, C., Li, B., Liu, M., Boning, D., and Hsieh, C.-J. (2020). Robust deep reinforcement learning against adversarial perturbations on state observations. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Zou, A., Wang, Z., Carlini, N., Nasr, M., Kolter, J. Z., and Fredrikson, M. (2023). Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*.
- Zügner, D., Akbarnejad, A., and Günnemann, S. (2018). Adversarial attacks on neural networks for graph data. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, pages 2847–2856.